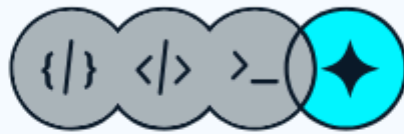


Testing

Team 8: Ben and Bensons



B&B

CODE. CREATE. CONNECT.

Team Members: Alyssa Skipper, Benjamin Senior, Benson Chow, Chloe Ward, Florian Mengkris, Hannah Thomas, James Ingram, Olivia Spencer

Testing Methods and Approaches

Our testing methods were devised with completeness, correctness and appropriateness with respect to the original requirements document as a priority. In particular, tests were designed to ensure that functionality is relevant to the requirements, that it meets all of the necessary requirements, and that it is accurate in doing so.

Before making any changes to implementation, we first created a series of regression tests to ensure that the desirable properties of the original implementation are not lost, reverting changes as soon as any issues arise. From there, we then iteratively used the requirements document to look for behaviours that the program should exhibit, generating black box test cases using both interior and boundary inputs from different partitions of the input space. This was done case-by-case for all of the relevant tests, with at least one representative for all equivalence classes of inputs including different types of invalid inputs. Where applicable, these cases were tested via parametrised unit tests to make the tests less brittle.

In broad terms our testing methods are split between three levels of scope: “unit” tests that focus on more self-contained components, “integration” tests that focus on how components interact, and broad “system” tests that focus on the larger functionality of the software. As advised by Google’s book[1, p. 247], we have aimed to split these tests into a ratio of roughly 80% (unit) to 15% (integration) to 5% (system), but this caused difficulties since a large amount of refactoring was needed to encourage more self-contained components. To reduce this and avoid the “God class” antipattern, we used a sociable[2] unit test approach, but this had limitations as it resulted in test code being more extensive than desired.

Our tests were conducted in two different ways where appropriate: most tests were automated, and run through GitHub Actions on every Git push; and the remaining tests were executed manually using more heuristic, manual methods. We used both knowledge of the code and a coverage checker as guidance to produce white box tests in addition to the tests already used. This approach made it simple to prioritise test coverage in components with more priority. To achieve an optimal level of test automation, we utilised LibGDX’s headless backend and Mockito: these allow for tests which depend on a LibGDX environment to run without rendering graphics, and allow simulation of inputs such as key-presses. Several vital methods initially failed in the headless environment due to dependence on the rendering backend, meaning that we needed to refactor much of the code and introduce test doubles to replace the graphical components with minimal effect on the remaining game logic.

The refactoring involved separating the dependencies of the methods from any graphical components and containing those components within new “delegate” classes that could be replaced with stubs during testing using a fake graphics manager class. We also used dependency injections[1, p. 288] to make the use of these stubs and fakes more convenient. This fake graphics manager is then introduced through a fake override of the Main class, designed to minimise the amount of testable game logic which is overridden by automated tests. As expected, this resulted in said tests being easier to set up in parallel with changes to the codebase. Since the graphical components are significant for the functionality of the game, we also designed manual integration and system tests to ensure that the system is able to meet its requirements more broadly without any test doubles in use.

Testing Report (Key: Success, Partial success, Failure)

Automated Tests

Unit Test Names	Test Description and Results	Requirements
testAssets	Checked that each asset file (test parameter) exists according to LibGDX, including the UI skin.	FR_VISUALS, NFR_MENU_USE
timerDecrementTest	Checked that the timer correctly changes given a series of valid inputs, and throws an error if an invalid input (such as a negative increment) is given.	FR_CLOCK
testInRange	Checked that a collection of valid positions are correctly identified as being in range of a nearby object, subject to an offset.	FR_KEYCARD, FR_DRINK, FR_BONUS
gameVolumeTest1, musicVolumeTest1	Checked that a collection of illegal game and music volumes (that is, outside the interval [0,1]) cause illegal argument exceptions errors in the audio manager.	FR_SETTINGS, FR_MUSIC, FR_VOLUME_ADJUST,
gameVolumeTest2, musicVolumeTest2	Checked that a collection of valid game and music volumes (that is, in the interval [0,1]) including the minimum (0) and maximum (1) are accepted by the audio manager.	FR_SETTINGS, FR_MUSIC, FR_VOLUME_ADJUST
playButtonTest	Checked that the play button on the instructions and pause screens respond to clicks and change the screen to the game.	FR_RETURN, FR_PAUSE
torchTest	Checked that the torch light is toggled correctly when the torch is toggled.	FR_TORCH
upTest, downTest, leftTest, rightTest	Checked that the player moves correctly according to their speed when WASD and the arrow keys are used and that the correct animations are used for each key.	FR_KEYBOARD_CONTROL, FR_VISUALS
mapLimitTest	Checked that the player cannot move beyond the height and width limits of the map.	FR_MAP_LIMIT
puddleCollisionTest	Checked that the collisions and non-collisions between puddles and points in space can be correctly detected by the puddle object.	FR_PUDDLE
character1SelectionTest, character2SelectionTest	Checked that, when a character is selected, the world is provided the correct assets and the screen is changed to the instructions screen. For the male and female options during the tests, the world used the correct assets and the instruction screen is set as the current screen.	FR_CHARACTER_SELECTION, FR_INSTRUCTIONS_MENU
scoreTest	Checked that the score correctly updates with different appropriate arguments, including changes of 0 and negative changes, without becoming negative.	FR_SCORE
pveEventTest, nveEventTest, hdnEventTest	Checked that a single positive event, a single negative event or a single hidden event is enough to obtain the event achievement. During the tests, it was found that following any single event of any type, the event achievement is provided.	FR_ACHIEVEMENTS
completedNoEventTest	Checked that, if no events have occurred, then the event achievement is not given. During the test this was found to be the case.	FR_ACHIEVEMENTS
hotbarTest	Checks that items' images are shrunk down to hotbar scale when collected.	FR_HOTBAR
completedAllEventsTest	Checked that, if all events are completed, then the "all events" achievement is given. When the maximum total of events occurred, the achievement was given.	FR_ACHIEVEMENTS
notCompletedAllEventsTest	Checked that, if not all events occur, then the "all events" achievement is not given. For numerous combinations of events, the achievement is correctly not given.	FR_ACHIEVEMENTS

finishFastAchievementTest	Checked that, if the game is completed fast enough, then an achievement is given. When the game was completed in half the time, the achievement was given.	FR_ACHIEVEMENTS
finishFastNoAchievementTest	Checked that, if the game is not completed fast enough (that is, if more than half the time limit is taken), then an achievement is not given for fast completion.	FR_ACHIEVEMENT
onLeaderboardTest,notOnLeaderboardTest	Checked that an achievement is given for being on the leaderboard if and only if the player is on the leaderboard. During testing, when this was the case, the achievement was given, but when there were 10 better scores than the player's, it was not given.	FR_ACHIEVEMENT
rainStopTest, rainStayTest	Checked that the rain stops after its duration has ended, and does not stop beforehand. Across multiple different durations, this was found to be the case.	FR_RAIN
gooseDeletesWormTest	Checked that, when a worm is requested for pickup and the goose targets, the goose deletes the worm.	FR_WORM, FR_GOOSE
drinkActiveTest	Checked that energy drinks' effects are active for their duration after being collected.	FR_DRINK
drinkDoneTest	Checked that energy drinks' effects expire after their duration has completed.	FR_DRINK
drinkSpeedTest	Checked that the existence of an active energy drink (where the existence of such a drink is simulated by a mock for the sake of isolation) causes speed increase.	FR_DRINK
gooseFoodTest	Checks that baby geese are created when the goose food is used on the goose.	FR_HIDDEN_GOOSE_EVENT
stealTorchTest	Checks that the torch switches from the player to the goose when stolen by the goose.	FR_GOOSE

Integration Test Names	Test Description and Results	Requirements
testGameVolumeSliders, testMusicVolumeSliders	Checked that, when volume sliders move (including to the endpoints), the audio manager's game and music volumes update correspondingly.	FR_SETTINGS
pausedTimerTest, timerTestGame, timerTestRC, timerTestLW	Checked that the timer decreases correctly on the game, Langwith and Ron Cooke screens (using simulated render deltas), and does not change when the game is paused.	FR_PAUSE, FR_PAUSE_COUNTDOWN
leaderboardAddTest	Checked that the leaderboard is told to add the correct username-score pair when the win screen is opened for numerous different scores.	FR_LOCAL_SCORE
puddleInteractionTest	Checked that the game detects when a player collides with a puddle.	FR_PUDDLE
rainPuddleTest, noRainPuddleTest	Checked that puddles may be added whilst it is raining, but not if it is not. During the tests, this was precisely the case.	FR_RAIN, FR_PUDDLE
wormCollisionTest, wormNoMoveTest	Checks that, when a player is close enough to a worm, a collision is detected; and that, following a collision, the player is stopped from moving.	FR_WORM
outOfTimeTest	Checked that the game is lost (that is, the game over screen opens) when 5 minutes or more passes on the timer on the game screen.	FR_COUNTDOWN, FR_END_SCREEN, NFR_GAME_END
ptTest1, ptTest2	Checked that the player score is increased upon completion of bonus events such as the goose stealing the torch and the goose being fed.	FR_INTERACTION_BONUS

Manual Tests

Below we have summarised the manual tests carried out for the game: for further details including evidence, see the manual testing report on the project's website. These tests were necessarily performed manually in order to accurately capture the performance of the game in the same environment as the player would use; to check that effects such as collisions and visuals are consistent with what the user experiences; and to ensure that the game is intuitive and well-designed for humans to enjoy.

Unit Test Name	Test Summary	Requirements
MT_Collision	Checked that the player correctly collides with boundaries on the map.	FR_COLLISION
MT_Pause_Time	Checked that the time taken to pause after P is pressed is less than 1 second.	NFR_PAUSE_MENU
MT_Input_Delay	Checked that the input delay on player movement does not exceed 1ms.	NFR_PLAYER_RESPONSE

Integration Test Name	Test Summary	Requirements
MT_Start_Time	Checked that the time to get from the menu to the game is less than 8 seconds.	NFR_MENU_TO_GAME , FR_MOUSE_CONTROL
MT_Load	Checked that the time taken for the game map to load and unload when entering and exiting buildings is no more than 1 second.	NFR_MAP_LOAD
MT_Inter-Menu	Checked that the time to load between any pair of menus is at most 16ms.	NFR_INTER-MENU

System Test Name	Test Summary	Requirements
MT_Frame_Rate	Checked that the frame rate is sufficiently high on average at all times.	NFR_GAMESPEED, NFR_CONSISTENT_GAMESPEED
MT_Resize	Checked that the game's graphics rescale correctly with the window.	FR_RESIZE
MT_Navigation	Checked that it is possible to navigate across the map in less than 2 minutes and 30 seconds.	NFR_NAVIGATION, FR_KEYBOARD_CONTROL
MT_Memory	Checked that the game's memory usage does not exceed 500MB.	NFR_GAME_SIZE
MT_Min_Time	Checked that the game is not easily beaten in 2 minutes or less.	NFR_MIN_TIME
MT_OS_Compatibility	Checked that the game runs without OS-specific errors on three different OSs: Windows 11, Ubuntu LTS, and MacOS.	NFR_OS_VERSIONS
MT_Accessibility	Checked that the game is legible with a greyscale filter applied.	NFR_ACCESSIBILITY, NFR_MENU_USE

Conclusion & Summary

With these tests, we verified the correctness and completeness of the program with respect to most of the necessary ("shall") requirements. The remaining requirements were checked via user evaluation, and these were slightly less successful from a user perspective. This

indicated that perhaps the testing approach should have included more external testers who were able to give their input more frequently, allowing for a better UX earlier in development.

Testing Material Links

Manual Testing Report

https://stage-2.eng-1.bensonc.how/assets/pdfs/manual_testing_report.pdf

Automated Test Summary Report

https://stage-2.eng-1.bensonc.how/test_report/index.html

Jacoco Automated Test Coverage Report

https://stage-2.eng-1.bensonc.how/coverage_report/index.html

References

- [1] T. Winters, T. Manshreck, and H. Wright, *Software Engineering at Google: Lessons Learned from Programming Over Time*. "O'Reilly Media, Inc.," 2020.
- [2] M. Fowler, "Unit Test," martinowler.com. Accessed: Jan. 07, 2026. [Online]. Available: <https://martinfowler.com/bliki/UnitTest.html>